

Report for the Course on Cyber-Physical Systems and IoT Security

Parrot, a software-only anti-spoofing defense system for the CAN bus

Gianluca Bresolin

Objectives

In this report, the results of a Rust implementation of the Parrot anti-spoofing defense system for the CAN bus are presented.

As described in the original paper, the Parrot system relies on the victim ECU being equipped with a CAN controller capable of sustaining rapid frame transmission (with a minimum inter-frame gap of $\sim 3\mu\text{s}$), allowing it to apply the counter-attack defense autonomously, driving the attacker ECU into the bus-off state. This is the scenario implemented and evaluated in this report

The objective of this report is to demonstrate how our Rust implementation of the Parrot system can effectively drive into bus-off a compromised ECU, within a CAN bus environment where each ECU is pre-configured with a list of its own unique message IDs.

Regarding the threat model, we consider an attacker capable of transmitting any legal frame, masquerading as any message ID. Furthermore, the compromised ECU is assumed to have full semantic understanding of the CAN bus communication, including the contents and structure of legitimate messages.

However, we assume that the attacker cannot change the ECU's CAN controller low-level behavior, which is typically implemented in hardware.

To demonstrate the effectiveness of this implementation, a simple scenario involving three ECUs is emulated: one victim ECU, one attacker ECU, and one bystander ECU that reacts to the received frames with its own behaviors, allowing observation of how normal bus traffic is affected once the attacker begins spoofing the victim's legitimate frames, until the attacker is driven to bus-off by the victim engaging the Parrot defense.

The remainder of this report is organized as follows: the '*System Setup*' section describes the simulation tools used and implemented for the experiments; the '*Experiments*' section presents the methodology adopted to carry out our experiments. Finally, the '*Results and Discussions*' section outlines the results and our considerations regarding the experiments and the implemented defense system.

System Setup

To emulate each ECU, since we didn't have the possibility to test our implementation under real hardware, we decided to emulate our ECUs via the QEMU emulator of a *Stellaris LM3S6965 EVB* board, based on an ARM Cortex-M3 microcontroller, which is well known for being used as ECU in CAN Bus Protocol real application.

Working on an emulation forces us to operate in a bare metal `#[no_std]` environment where, due to the strict memory and hardware constraints of embedded microcontrollers, we

cannot rely on OS commodities available on general-purpose systems, which is the main objective of adopting QEMU for this project.

Even if QEMU offers a CAN Bus protocol support, its emulation does not enable us to express the error granularity ‘to the bit’ required for this experiment and the Parrot Defense System. For this reason, we decided to implement a virtual can controller on our own. This was done by forking the official [QEMU repository](#) and adding a `virtual_can_controller.c` as a *SysBusDevice* MMIO.

In QEMU, a *SysBusDevice* is the standard abstraction used to model hardware peripherals that communicate with the CPU through *Memory-Mapped I/O* (MMIO). This means that the device’s internal registers are mapped to specific physical memory addresses. The CPU interacts with the device by simply reading and writing to those addresses, exactly as it would with ordinary RAM. With *SysBusDevices*, QEMU intercepts these accesses through two callbacks, `mmio_read` and `mmio_write`, and uses them to simulate the hardware behavior in software. This is precisely how real CAN controllers (such as the *SJA1000 CAN Controller*) work in embedded systems.

Our `virtual_can_controller.c` replicates the register interface, exposing to the guest device the same memory-mapped register as a real CAN controller would, while internally implementing the CAN protocol behavior, with errors reflecting a precision down to single-bit transmission granularity.

For the purposes of this project, and in order to avoid introducing unnecessary additional complexity, CRC computation, ACK handshaking and Extended (29-bit) identifier support were not needed and were therefore simplified: the CRC field is always written as a sequence of 15 1 bits and is not verified, the ACK slot is hardcoded to 1, and only the Standard 11-bit identifier format is supported, while keeping IDE and `r0` fixed to 1.

Specifically, when the guest ECU wants to transmit a CAN frame, it does so through a sequence of MMIO writes, each targeting one of the exposed registers: ID (0x00), RTR (0x04), DLC (0x08), Data-low (0x0C) and Data-high (0x10). Each write updates the corresponding field of a `RoughCANFrame`, a structure that holds the frame data in its plain and unstuffed form, together with an `is_ready` flag for each field. These flags let the controller determine when a `RoughCANFrame` is fully populated and therefore ready to be transformed into a `CANFrame` structure.

Once all fields have been written, the ECU triggers the actual send by writing value 1 (send frame) in the `Command register` (0x14): this invokes `send_rough_can_frame()` which checks that every ready flag is set and, if so, calls `build_can_frame()`, which serializes the `RoughCANFrame` into the final bit-level representation.

More precisely, it writes the *Start-of-Frame* dominant bit, the 11-bit *identifier*, *RTR*, the *IDE* and *r0* control bits, the 4-bit *DLC*, and the *data* bits, followed by a 15 bits sequence of 1 as CRC bits (as previously mentioned). Over this region is then run `apply_bit_stuffing()` to insert stuff bits wherever five consecutive identical bits occur, and finally the *ACK* slot/delimiter and the 7-bit *End-of-Frame* are appended unstuffed. All these bits are stored inside `CANFrame.bits`, an array of `uint8_t`. This choice is what actually enables bit-level granularity: rather than packing the frame into a compact bitfield or byte-aligned buffer, each single bit to be transmitted is kept as its own `uint8_t` element, so every bit of the frame

can be individually indexed, inspected, stuffed, or compared against the bit read back from the bus, without needing any bit-masking or shifting to isolate it.

Once the `CANFrame` is correctly built, it is pushed onto `tx_queue` via `append_can_frame_to_tx_queue()`.

Since a QEMU emulator can sustain only one device simulation at time, each ECU in our experiment runs inside a Docker Container, orchestrating them with Docker Compose (further details for the experiment configuration are provided in the '*Experiments*' section). Since sharing memory between ECU containers, while technically possible, is not straightforward and adds unnecessary complexity, we opted instead to simulate the *CAN bus* by implementing a *Virtual CAN Bus* as a standalone Rust TCP server, acting as the shared medium that all `virtual_can_controller.c` instances connect to over TCP, with one connection per ECU. This choice was primarily driven by simplicity and easier debugging, since a TCP-based medium keeps each ECU container fully isolated and makes it straightforward to observe, log, and reason about the bit exchange on the bus.

On startup, the server spawns a listener task on port 8080 that accepts incoming connections and assigns each one an incremental id. For every connected controller, two dedicated tasks are spawned: a *reader* task, which continuously reads one byte at a time from the socket (each byte representing a single transmitted bit, 0 or 1) and stores it in a shared `received_bits` map keyed by controller id; and a *writer* task, fed through an *MPSC* channel, which forwards the computed bus bit back to that controller. This per-bit, per-connection design allows the bus to operate at the same single-bit granularity as the controllers themselves.

In parallel, a main loop periodically computes the bus state and closes each "bit round": it waits until all currently registered controllers have submitted a bit and then it derives the resulting bus bit by emulating the wired-AND behavior of the physical CAN bus. The resulting bit is broadcast back to every registered controller, the `received_bits` map is cleared, and the loop repeats.

It is important to note that QEMU, being an event-driven emulator, does not provide a real hardware timer: temporal ordering of events within a single emulated instance is respected, but as soon as synchronization with other external emulated instances is required, achieving reliable timing alignment becomes unreliable and difficult. Furthermore, QEMU emulation does not aim to reflect the actual computational power and execution timing of real hardware, since its aim is to be a tool to verify that the code functions correctly on the target machine, respecting its hardware and software constraints, and not to provide accurate timing. For this reason, the *Virtual CAN Bus* was designed to enforce synchronization through a model that waits for all connected ECUs to have transmitted their bit, rather than computing the bus state every μs . This design choice is also why the TCP-based communication of each transmitted bit between a CAN controller and the *Virtual CAN Bus*, with all the overhead that TCP entails, does not represent a problem for synchronization or timing.

This is admittedly a strong assumption, but an unavoidable one for this testing scenario, and one that should be considered in real-world applications.

From this choice, every TX bit originates from an RX bit, meaning that each controller only transmits its next bit *after* having received the previous resolved bus bit from the *Virtual CAN Bus*. As a consequence, no controller can "run ahead" of the others, because its own transmission is always triggered by, and therefore paced by, the bus's response to the previous round.

Turning back to the `virtual_can_controller.c`, when a bit arrives from the socket, it is handled by `rx_callback()`: this function is responsible for detecting whether the reception of a new frame has started, recognizing a dominant *Start-of-Frame* bit while no reception is already in progress, and, in that case, instantiating a new `CANFrame` in `rx_queue`.

It then checks, if a bit was transmitted, whether the received bit matches what was sent, through `check_transmission()`. Specifically, this function detects arbitration losses or transmission errors on a bit-by-bit basis, ensures that error flags are sent correctly, and detects eventual form errors, such as unrespected *error-delimiter* or *IFS* (which play a central role within the *Parrot* defense algorithm).

Once transmission is eventually checked, it is then applied the bit unstuffing logic via `apply_bit_unstuffing()`, which detects possible *bit-stuffing* errors, discards stuffing bits or appends the actual frame bit to the last `CANFrame` appended in `rx_queue` with `append_rx_bit_to_can_frame()`, which also checks whether the reception has ended by comparing the last 7 received bits against a sequence of 1s.

Once a `CANFrame` is fully received, it is marked as complete and the RX Interrupt (IRQ 40 in our implementation) is raised, notifying the ECU that a new frame is ready to be read through the MMIO registers.

Finally, `tx_callback()` is called at the end of the `rx_callback()`. This function first checks whether there are *error flag* bits to transmit, then checks whether *error-delimiter* or *IFS* bits still need to be sent, and then checks whether `tx_queue` is not empty: if so, and if transmission is allowed (i.e. no other reception is in progress), it sends the bit at `tx_bit_cursor` of the `CANFrame` at index 0 of the `tx_queue` over the TCP socket to the *Virtual CAN Bus*, one byte per bit, incrementing the cursor after each successful bit until the whole stuffed frame (including *EoF*) has been shifted out. Note that keeping this cursor as an explicit index, rather than consuming the frame destructively, makes it trivial to reset a transmission: it is enough to set `tx_bit_cursor` back to 0 to restart sending the same frame from its first bit.

Regarding the *ECU CAN bus state*, this is maintained and updated through a small set of dedicated functions: `apply_tx_error()` and `apply_rx_error()` are invoked whenever a transmission or reception error is detected respectively, incrementing the *TEC* by 8 or the *REC* by 1, in line with the standard CAN error-counting rules. Conversely, `apply_successfull_rx()` and `apply_successfull_tx()` are called after a successful reception and/or transmission, decrementing *TEC* and *REC* respectively by 1. After every such update, `update_can_bus_state()` is invoked to re-evaluate the state of the controller, based on the current counter values, transitioning it between `ERROR_ACTIVE`, `ERROR_PASSIVE` and `BUS_OFF`. This last one, once reached, is permanently locked, with no further updates and transmissions allowed.

Experiment

The experiment used to validate the *Parrot* defense algorithm consists in a multi-node scenario, with the whole system orchestrated with Docker Compose, spinning up four containers connected over a single bridge network (`can-net`), which models the shared physical CAN bus at the network topology level.

One node is represented by the `virtual_can_bus`, which runs the Rust TCP server described in the previous section, emulating the behavior of the physical medium.

The remaining three containers are instead the ECUs, built from the same image but instantiated with different build arguments. Specifically, each ECU requires to define `MY_IDS`, `INTERESTED_IDS` and `ROLE` as `ENV:MY_IDS` specifies the CAN identifiers the ECU owns and is allowed to transmit under, `INTERESTED_IDS` specifies the identifiers it listens to and reacts upon reception, and finally `ROLE` is used to determine the application logic that the device needs to execute, allowing the same Docker image to be reused for different purposes.

container_name	MY_IDS	INTERESTED_IDS	ROLE
victim_device	"0x100"	""	"victim"
attacker_device	"0x100"	""	"attacker"
led_device	""	"0x100"	"none"

The table above reports testing configuration for those environment variables.

The testing scenario is built so that the `attacker_device` and the `victim_device` share the exact same CAN identifier (0x100), allowing the attacker to spoof the victim's identifier, reproducing the actual threat model the *Parrot* algorithm is designed to detect, namely a malicious node impersonating a legitimate one on the bus.

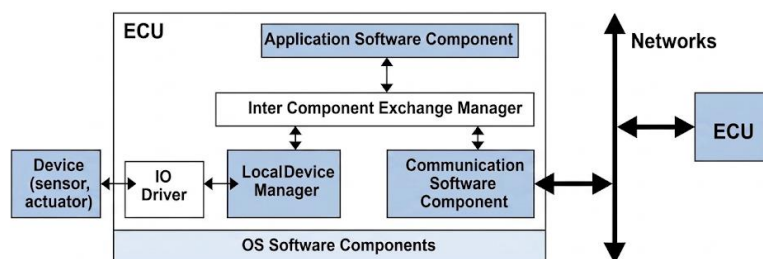
To observe the effects of an injected spoofed frame, the `led-device` is configured as a listen-only ECU: upon receiving a frame with ID 0x100 whose payload is not all-zero, it simulates turning on a virtual LED by printing "LED is turned ON" to the console. The all-zero payload is deliberately excluded from this trigger because it is reserved for the *Parrot* defense algorithm itself, used as the defensive message (`D_message`) rather than as legitimate application data, so that the LED reaction only reflects genuine (or spoofed) application traffic and not that of the defense mechanism.

All three ECU containers use `depends_on: virtual-can-bus` to ensure the bus server is up before any ECU attempts to connect and share the same `can-net` network so that TCP connectivity to the bus is possible while each ECU remains fully isolated in its own container.

Independently of its role, each emulated ECU follows the same internal software architecture on the Rust side, mirroring how real ECUs are structured. Specifically, the Rust code is organized in the following *crates*¹:

¹In Rust, a crate is the fundamental unit of compilation and code distribution, producing either an executable binary or a reusable library.

- **application_software_component**: this crate implements the ECU's actual application logic, based on the `ROLE` env value. At this level, the application logic is not aware of the underlying hardware details and only interacts with the `inter_component_exchange_manager`;
- **inter_component_exchange_manager**: this crate coordinates data exchange between the different software components internally, decoupling them from each other;
- **local_device_manager**: this crate handles potentially different *IO Drivers* (that need to implement the `DeviceDriver` trait), interacts directly with them, and exposes a hardware agnostic interface, so the latter never has to deal with hardware specifics;
- **communication_component**: this component is responsible for sending and receiving messages over the CAN bus, exposing a hardware agnostic interface to communicate, by directly interacting with MMIO registers connected to the (virtual) CAN controller.



The *Parrot* defense algorithm is implemented inside the **communication_component**, inside the `parrot` module, which is responsible for offering all the functions that interact with the shared state between the application process and the process on which *Parrot* runs in the background, in order to keep the ECU available for other tasks while the defense algorithm is executing. This shared state is protected using `cortex_m::interrupt::Mutex` and lock-free `AtomicBool/AtomicU8` variables with explicit `Acquire/Release/AcqRel` ordering. Specifically, the shared state consists in `SUSPECT_FOUND`, `COLLISION_DETECTED`, `TX_D_SIGNAL` and the `COLLISION_COUNT`, so that the interrupt-driven defense routine and the normal reception/application flow can safely read and update the same data without incurring in data races.

Detection starts in `communication_component::receive()`, which, after reading the newly completed frame out of the CAN controller's MMIO registers, calls `check_frame()` on every incoming frame before dispatching it to possible interested listeners.

```

pub fn check_frame(
    &mut self,
    frame: CANFrame
) {
    if self.my_ids[..self.my_ids_count].contains(&frame.id) {
        if !self.tx_flag && !parrot::defense_mode() {
            parrot::activate_defense_mode();
            parrot::set_attacker_frame(frame);
            cortex_m::peripheral::SCB::set_pendsv();
            return;
        }
        // else: it was our packet or defense mode already on
        if self.tx_flag && parrot::is_d_message(frame) {
            parrot::notify_d_message_received();
        }
    }
}

```

This function compares the frame's ID against `MY_IDS`: if the ID belongs to this ECU but the frame was not the one this ECU itself just transmitted (`!self.tx_flag`) and the defense is not already engaged (`!parrot::defense_mode()`), it concludes that another node on the bus has spoofed this ECU's identifier. It then calls `parrot::activate_defense_mode()`, stores the offending frame via `parrot::set_attacker_frame()`, and raises a PendSV exception through `SCB::set_pendsv()`.

Indeed, the process executing the *Parrot* defense algorithm runs at a lower interrupt priority, so that the ECU application logic and the CAN reception stay fast and non-blocking. This allows `parrot::engage()` to run its while loop logic, which keeps calling `parrot::transmit_nd_message()` as long as `SUSPECT_FOUND` is true and `!COLLISION_DETECTED`.

This last function implements, in turn, a bounded loop up to `bound`, initially set to `ND = 100` attempts): at each iteration it invokes `parrot::transmit_d_message()` to send a defensive message, which actually builds and sends the defensive message by calling `communication_component::send()`, reusing the same ID and DLC as the attacker's frame but with an all-zero payload, updates the shared state `D_MESSAGE` with these same values, and only at the end sets `TX_D_SIGNAL` to true, signaling that it is now awaiting reception acknowledgment for that specific frame.

After instantiating the frame transmission, the algorithm enters in a second while loop that waits, via `wfi()` (thus avoiding busy waiting), for `TX_D_SIGNAL` to return to false. This flag is precisely what notifies that the transmitted defensive message has been correctly acknowledged upon reception: the process handling frame reception, through `parrot::is_d_message()`, compares the received frame against what is stored in the shared `D_MESSAGE` and, on a match, calls `parrots::notify_d_message_received()` to bring `TX_D_SIGNAL` back to false, thereby unblocking the wait loop.

Once unblocked, it checks whether `COLLISION_DETECTED` was detected in the meantime.

This flag is set to true once the first 16 collisions have been identified following the transmission of a defensive messages that has collided with an attacker's frame on the spoofed identifier: this is made possible by an exception raised by the `virtual_can_controller.c` itself, which signals the detection of a transmission error on the CAN frame, and is handled by `communication_component::tx_error_handler()`, which in turn calls `parrot::collision_detected()` to increment `COLLISION_COUNT` and, once it reaches 16, set `COLLISION_DETECTED` to true.

If so, both `COLLISION_DETECTED` and `SUSPECT_FOUND` are reset to false and the attempt budget is extended (`bound = i + 16`), to keep probing for the attacker; otherwise, the loop simply moves to the next iteration, incrementing its iterator.

```
const ND: u8 = 100;

pub static PENDING_ENGAGE_FRAME: Mutex<RefCell<Option<CANFrame>>> = Mutex::new(RefCell::new(None));
pub static SUSPECT_FOUND: AtomicBool = AtomicBool::new(false);
pub static COLLISION_COUNT: AtomicU8 = AtomicU8::new(0);
pub static COLLISION_DETECTED: AtomicBool = AtomicBool::new(false);
pub static TX_D_SIGNAL: AtomicBool = AtomicBool::new(false);
pub static D_MESSAGE: Mutex<RefCell<Option<CANFrame>>> = Mutex::new(RefCell::new(None));

#[exception]
fn PendSV() {
    let frame = cortex_m::interrupt::free(|cs| {
        PENDING_ENGAGE_FRAME.borrow(cs).replace(None)
    });
    if let Some(frame) = frame {
        engage(frame);
    }
}

fn engage(attacker_frame: CANFrame) {
    hprintln!("DEFENSE MODE ON");
    while SUSPECT_FOUND.load(Ordering::Acquire) && !COLLISION_DETECTED.load(Ordering::Acquire) {
        transmit_nd_message(attacker_frame);
    }
}

fn transmit_nd_message(attacker_frame: CANFrame) {
    let mut bound = ND;
    let mut i = 0;
    while i < bound {
        transmit_d_message(attacker_frame);

        while TX_D_SIGNAL.load(Ordering::Acquire) {
            // wait for incoming frame interrupt before checking
            // (avoid busy wait)
            cortex_m::asm::wfi();
        }

        if COLLISION_DETECTED.load(Ordering::Acquire) {
            COLLISION_DETECTED.store(false, Ordering::Release);
            SUSPECT_FOUND.store(false, Ordering::Release);
            bound = i + 16;
        }
        i += 1;
    }
}
```

To conclude the scenario, the testing setup has the `attacker_device` ECU attempt to transmit two consecutive frames under the same ID assigned to the `victim_device` ECU (0x100), with both the `victim_device` and the `led_device` listening on the CAN bus. This should trigger the *Parrot* defense on the `victim_device` 's side upon reception of the first spoofed frame, while the second attacker frame is expected to fail transmission entirely, with the `attacker_device` ECU driven into `BUS_OFF`. The `attacker_device`, listening for the same ID (0x100), should therefore react only once, turning its virtual LED on solely in response to the first (successfully delivered) spoofed frame, and not to the second one, which never reaches the bus.

Results and Discussion

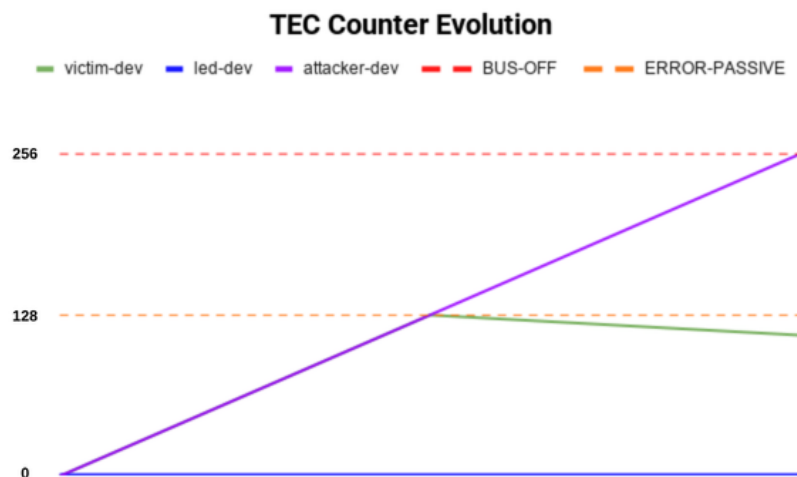
The project, including the full source code, is publicly available on *GitHub* at <https://github.com/GianlucaBresolin/ParrotCAN>.

To run the experiment, it is not necessary to clone the entire repository: it is sufficient to follow the instructions in [INSTRUCTIONS.md](#), which only require downloading a single [docker-compose.yaml](#) file, enabling cross-platform CPU architecture emulation and pulling the pre-built images from *Docker Hub* (<https://hub.docker.com/repositories/gianluocabresolin>). Depending on the desired level of debug output, three different testing profiles are available: default, debug-state, and debug-verbose.

Cloning the full repository is discouraged, as it includes a fork of the *QEMU* repository, which is quite heavy and can take a significant amount of time to download. To facilitate consultation of `virtual_can_controller.c` within this large repository, a direct link to the file within the fork is provided: [virtual_can_controller.c](#).

In this project, we successfully achieved the desired behavior. The testing scenario described above, where an `attacker_device` tries to transmit two consecutive spoofed frames under the `victim_device`'s ID (0x100), with the `victim_device` and `led_device` listening on the bus, was reliably reproduced, with the `victim_device`'s *Parrot* defense triggering on the first spoofed frame and driving the `attacker_device` into `BUS_OFF` before the second frame could be transmitted.

This confirms that, thanks to our `virtual_can_controller.c` together with the *Virtual CAN Bus* reproducing bit-level protocol semantics, we were able to verify the effectiveness of the *Parrot* algorithm as a software-only defense mechanism against malicious ECUs, able to detect and operate at runtime.



The victim ECU correctly identified an attacker impersonating its own CAN identifier, engaged the defense routine by repeatedly transmitting defensive messages, and drove the attacker into a `BUS_OFF` state, effectively silencing the spoofed traffic on the bus. This can be observed both through the single `led_device`'s reaction to the injected attacker frames in the terminal output produced during the execution of the test scenario. As a result of the testing scenario, it is possible to observe the evolution of the devices' `TEC` counters

throughout the experiment, as shown in the figure above. This data was observed and collected by running the `debug-state` test profile.

Despite its effectiveness, the *Parrot* defense algorithm presents a few limitations worth discussing. The defense algorithm appears vulnerable to consecutive attacks involving multiple attacker ECUs, if these occur before the victim ECU's TEC counter has time to decay back to 0, since repeated defense activations would compound on top of an already-elevated error count. Related to this, the algorithm implicitly assumes that the victim ECU's TEC starts at 0 when the defense is triggered, an assumption that is not always predictable nor guaranteed to hold in practice.

Regarding our implementation and testing scenario, there is an assumption that the victim ECU does not transmit other frames while the defense mode is active: this constraint is easily enforceable in principle, but it is not currently implemented in the codebase. Finally, as mentioned earlier, the Parrot defense requires tight synchronization between the attacker's and the victim's transmissions to generate the intended collisions, a condition that is desirable but not always guaranteed in real-world bus conditions.

Reference

Dagan, T., & Wool, A. (2016). *Parrot, a software-only anti-spoofing defense system for the CAN bus*. Tel Aviv University